EXPT NO: 10 DATE: 12/10/2026	BUILDING A SIMPLE LLM CHATBOT USING STREAMLIT/GRADIO

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is a Large Language Model (LLM)?
A Large Language Model (LLM) is an artificial intelligence model trained on large amounts of text data to understand and generate human-like language. It can answer questions, summarize documents, translate languages, write code, and generate content. LLMs learn patterns, relationships, and context from data during training. Examples include ChatGPT, Google Gemini, and Claude. They are widely used in chatbots, education, content creation, and programming assistance. However, their responses may sometimes contain incorrect or misleading information.

2. What is the role of the Transformer architecture in modern LLMs?
The Transformer architecture is the foundation of most modern Large Language Models. It uses a mechanism called self-attention to understand relationships between words in a sentence, even when they are far apart. This helps the model understand context and generate meaningful responses. Transformers can process multiple words in parallel during training, making them efficient for large datasets. They use components such as attention layers, feed-forward networks, and positional information. Examples of Transformer-based models include GPT, BERT, and T5.

3. What is a prompt, and how does it influence an LLM's response?
A prompt is the input or instruction given to a Large Language Model to generate a response. It may contain a question, command, context, or example. The quality and clarity of a prompt influence the accuracy, relevance, and format of the response. A specific prompt generally produces a more useful answer than a vague one. For example, asking "Explain machine learning in five simple sentences" provides clear instructions. Prompt engineering is the process of designing effective prompts to obtain better results from an LLM.

4. What is the difference between a chatbot and a conventional rule-based program?
A chatbot uses artificial intelligence and natural language processing to understand user messages and generate conversational responses. Modern AI chatbots can handle different questions and adapt their responses according to context. In contrast, a conventional rule-based program follows predefined rules and conditions to produce outputs. It cannot easily handle unexpected inputs beyond its programmed rules. For example, an AI chatbot can answer questions in different ways, while a rule-based chatbot typically provides fixed responses. AI chatbots are more flexible, whereas rule-based programs are more predictable.

5. What are Streamlit and Gradio used for?
Streamlit and Gradio are Python frameworks used to create interactive web applications, especially for machine learning and artificial intelligence projects. Streamlit helps developers build data dashboards, visualizations, and AI applications with minimal code. Gradio allows developers to create simple interfaces for machine learning models, enabling users to provide inputs and view outputs easily. Both frameworks support rapid prototyping and demonstration of AI projects without requiring extensive web development knowledge. For example, Streamlit can display data analysis dashboards, while Gradio can provide an interface for an image classification model.

AIM

To build a simple chatbot or AI application using a Transformer-based Large Language Model (LLM), process user prompts, generate responses, and deploy the application using Streamlit or Gradio.

SOFTWARE REQUIREMENTS

- Python
- Google Colab / Jupyter Notebook / Local Python environment
- Hugging Face Transformers
- PyTorch (or another supported model backend)
- Streamlit or Gradio
- GitHub (for version control and submission)
- A web browser
- An LLM API key, if using a hosted model API

OBJECTIVES

Students will be able to:

- Understand the basic working of Transformer-based LLMs.
- Select and load a suitable pretrained language model or connect to a hosted LLM API.
- Accept user input through a prompt-based interface.
- Generate and display responses from the selected model.
- Build a simple chatbot interface using Streamlit or Gradio.
- Maintain conversation history where supported.
- Test the chatbot with different prompts and evaluate the relevance of its responses.
- Run and demonstrate the deployed application.

PROCEDURE

PART A: ENVIRONMENT SETUP AND MODEL SELECTION

Perform the following tasks:

- Create a Python notebook or project folder.
- Install the required libraries, such as transformers, torch, and either Streamlit or Gradio.
- Choose a suitable pretrained instruction-following or conversational model or select a hosted LLM API.
- Read the model documentation and check its hardware, memory, access, and usage requirements.
- Load the model and tokenizer or configure the API client and store the API key securely as an environment variable.
- Test the setup with one simple input prompt.

PART B: PROMPT HANDLING AND RESPONSE GENERATION

Perform the following tasks:

- Define a function that accepts a user message as input.
- Prepare the prompt in the format required by the selected model or API.
- Pass the prompt to the model and generate a response using suitable generation settings.
- Decode and display the generated response clearly.
- Handle empty input and basic runtime or API errors.
- Test the function using at least five different prompts and record the responses.

PART C: BUILDING THE CHATBOT INTERFACE

Perform the following tasks:

- Choose either Streamlit or Gradio to create the user interface.
- Add a text box or chat input for entering user messages.
- Display the user message and the model-generated response in the interface.

- Maintain conversation history if supported by the selected model and interface.
- Add a clear-chat or reset option if appropriate.
- Run the application locally and test the interaction from a web browser.

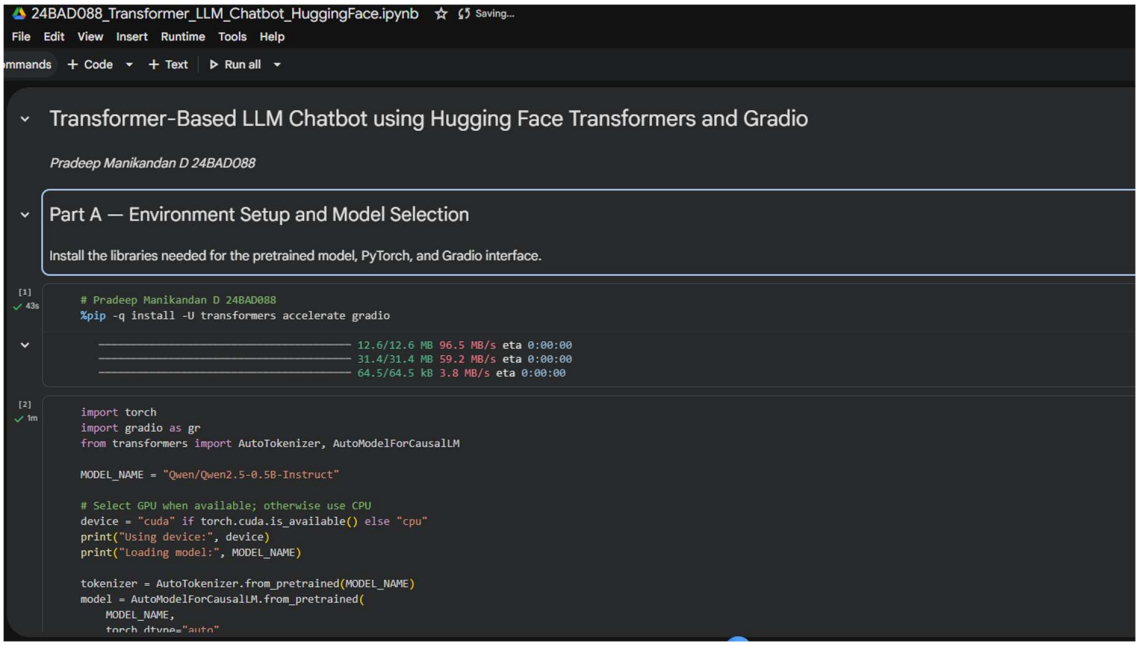
PART D: TESTING AND DEPLOYMENT

Perform the following tasks:

- Test the chatbot with factual, conversational, and task-oriented prompts.
- Check whether the responses are relevant, readable, and consistent with the prompts.
- Identify any incorrect, biased, or unsafe responses and note the limitations.
- Deploy the application using an available hosting option supported by Streamlit or Gradio or demonstrate it locally if deployment access is unavailable.
- Do not upload API keys or other secrets to GitHub; use environment variables or the hosting platform's secrets settings.
- Save the source code, requirements file, screenshots of the interface, and sample outputs.
- Include the GitHub repository link and the application link, if publicly deployed, in the submission.

EXPECTED OUTCOME

Students will understand the basic use of Transformer-based LLMs for prompt-based text generation and chatbot development. They will build a working chatbot interface using Streamlit or Gradio, test its responses with different prompts, and run or deploy the application. Students will also identify limitations such as inaccurate responses, model-access requirements, latency, and responsible handling of user inputs and API credentials.





Part B — Prompt Handling and Response Generation

The function below accepts a prompt, includes recent conversation history, generates a response, checks for empty input, and catches errors.

```
SYSTEM_PROMPT = (
    "You are a helpful AI chatbot for a student. "
    "Answer in simple, clear language. If you are unsure, say so."
)
```

```
        messages = [{"role": "system", "content": SYSTEM_PROMPT}]

        # Include recent conversation history when available
        if history:
            for item in history[-6:]:
                if isinstance(item, dict):
                    role = item.get("role")
                    content = item.get("content", "")
                    if role in ("user", "assistant") and isinstance(content, str):
                        messages.append({"role": role, "content": content})
                elif isinstance(item, (list, tuple)) and len(item) == 2:
                    user_text, assistant_text = item
                    if user_text:
                        messages.append({"role": "user", "content": str(user_text)})
                    if assistant_text:
                        messages.append({"role": "assistant", "content": str(assistant_text)})

        messages.append({"role": "user", "content": message.strip()})

        # Format messages using the model's chat template
        inputs = tokenizer.apply_chat_template(
            messages,
            tokenize=True,
            add_generation_prompt=True,
            return_tensors="pt",
            return_dict=True,
            truncation=True,
            max_length=2048
        )
        inputs = {key: value.to(device) for key, value in inputs.items()}

        with torch.no_grad():
            output_ids = model.generate(
                **inputs,
                max_new_tokens=120
```

```
                max_new_tokens=120,
                do_sample=True,
                temperature=0.7,
                top_p=0.9,
                repetition_penalty=1.1,
                pad_token_id=tokenizer.eos_token_id
            )

        # Decode only the newly generated tokens, not the prompt
        prompt_length = inputs["input_ids"].shape[-1]
        response = tokenizer.decode(
            output_ids[0][prompt_length:],
            skip_special_tokens=True
        ).strip()

        if not response:
            return "Sorry, I could not generate a response. Please try another prompt."

        return response

    except Exception as error:
        print("Chatbot error:", error)
        return "Sorry, an error occurred while generating the response. Please try again."

print("Chatbot response function is ready.")
```

```
Chatbot response function is ready.
```

```
test_prompts = [
    "What is artificial intelligence?",
    "Explain Python in simple words.",
    "What is a Transformer in machine learning?",
    "Give me three tips to prepare for an exam.",
    "Write a short motivational message for a student."
]
```

```
[5]
✓ 2m
for number, prompt in enumerate(test_prompts, start=1):
    answer = chatbot_response(prompt, history=[])
    print(f"TEST {number}")
    print("Prompt:", prompt)
    print("Response:", answer)
    print("-" * 70)
```

```
TEST 1
Prompt: What is artificial intelligence?
Response: Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that process information and perform tasks that typically require human intelligence, such as visual perception, speech recogn
----------------------------------------------------------------------
TEST 2
Prompt: Explain Python in simple words.
Response: Sure! Let's break down Python in a way that's easy to understand:

1. **Syntax**: Python uses indentation to define code blocks. It starts with an `#` symbol followed by the name of the block and then other lines.

2. **Variables**: Variables store values we can use later. They start with an `=` sign and can be any kind of data (like numbers or strings).

3. **Data Types**: Python has built-in data types like `int`, `float`, `str`, etc., which represent different kinds of information.

4. **Control Structures**: These
----------------------------------------------------------------------
TEST 3
Prompt: What is a Transformer in machine learning?
Response: A Transformer is a type of neural network model used in machine learning and natural language processing (NLP). It's named after the paper "Attention is all you need" by Vaswani et al. Transformers have been partic

Key features of a Transformer include:

1. **Self-Attention**: This allows the model to learn context from its inputs.
2. **Multi-head attention**: Enables each part of the input to be processed independently while still being able to look ahead.
3. **Encoder-decoder architecture**: The model can take an encoder output and
----------------------------------------------------------------------
TEST 4
Prompt: Give me three tips to prepare for an exam.
Response: Sure! Here are three tips to help you prepare for an exam:

3. **Join a Study Group**: Sometimes, discussing problems with classmates or friends can provide new insights and strategies
----------------------------------------------------------------------
TEST 5
Prompt: Write a short motivational message for a student.
Response: Certainly! Here's a concise motivational message for a student:

"Stay true to your dreams and never give up on your goals. Your efforts will not go unnoticed, and with hard work and determination, you'll achieve great things!"
----------------------------------------------------------------------
```

```
print("Empty input test:")
print(chatbot_response("   "))

sample_history = [
    {"role": "user", "content": "My name is Sam."},
    {"role": "assistant", "content": "Nice to meet you, Sam."}
]

print("\nConversation history test:")
print("Prompt: What is my name?")
print("Response:", chatbot_response("What is my name?", history=sample_history))
```

```
Empty input test:
Please enter a message.

Conversation history test:
Prompt: What is my name?
Response: Your name is Sam.
```

Part C — Build the Chatbot Interface

Run the following cell to create an interactive Gradio chatbot. It displays messages, passes conversation history to the response function, and provides a clear-chat control.

```
[9]
✓ 1s
demo = gr.ChatInterface(
    fn=chatbot_response,
    title="Transformer-Based AI Chatbot",
    description=(
        "A simple chatbot powered by Hugging Face Transformers and "
        "Qwen2.5-0.5B-Instruct. Enter a prompt below to begin."
    ),
    chatbot=gr.Chatbot(height=420),
    textbox=gr.Textbox(
        placeholder="Type your message and press Enter...",
        label="Your message"
    ),
    examples=[
        "What is artificial intelligence?",
        "Explain Python in simple words.",
        "What is a Transformer model?",
        "Give me three study tips."
    ]
)

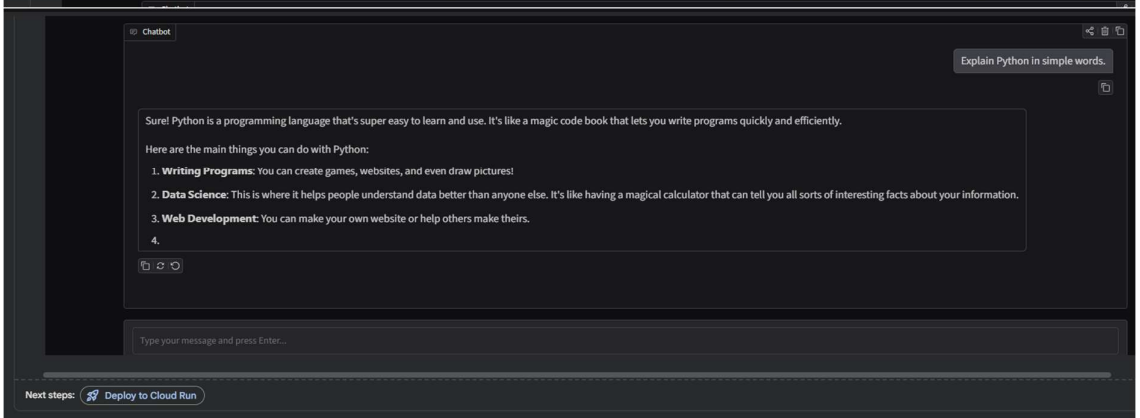
print("Gradio interface created. Launch it in the next cell.")

... Gradio interface created. Launch it in the next cell.
```

```
[10]
✓ 2s
demo.launch(share=True)

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://a45fcb00911435863b.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Huggi
```



Part D — Testing, Evaluation, and Deployment

Part D — Testing, Evaluation, and Deployment

Use the next cell to run a small functional test. Manually review each answer for relevance, correctness, readability, and consistency. A response being generated does not guarantee that it is factually correct.

```
[11]
✓ 1m
evaluation_cases = {
    "Factual": "What is machine learning?",
    "Conversational": "Hello! How are you?",
    "Task-oriented": "List three ways to manage study time."
}

for category, prompt in evaluation_cases.items():
    answer = chatbot_response(prompt, history=[])
    print("Category:", category)
    print("Prompt:", prompt)
    print("Response:", answer)
    print("Manual review needed: Check accuracy and relevance.")
    print("-" * 70)

Category: Factual
Prompt: What is machine learning?
Response: Machine learning is a type of artificial intelligence that enables computers to learn and improve from experience without being explicitly programmed. It involves using algorithms to analyze data, identify patterns
Manual review needed: Check accuracy and relevance.
----------------------------------------------------------------------
Category: Conversational
Prompt: Hello! How are you?
Response: Hello! I'm just a computer program, but it's been a while since we last met. I am not feeling well today and I hope you're doing well too. Is there something specific you'd like to talk about or ask me? I'm here to
Manual review needed: Check accuracy and relevance.
----------------------------------------------------------------------
Category: Task-oriented
Prompt: List three ways to manage study time.
Response: Certainly! Here are three effective ways to manage your study time:

1. **Prioritize Tasks**: Start by identifying what tasks or subjects need immediate attention. Focus on those that seem most important and urgent.

2. **Set Realistic Goals**: Break down larger projects into smaller, manageable tasks. This helps keep you motivated and reduces the feeling of being overwhelmed.

3. **Use Effective Study Techniques**: Utilize techniques such as summarizing information, creating mind maps, or using flashcards. These methods can make studying more efficient and less stressful.
Manual review needed: Check accuracy and relevance.
----------------------------------------------------------------------
```

```
[12]
✓ 0s
requirements_text = """torch
transformers
accelerate
gradio
"""

with open("requirements.txt", "w", encoding="utf-8") as file:
    file.write(requirements_text)

print("requirements.txt created.")
print(requirements_text)

requirements.txt created.
torch
transformers
accelerate
gradio
```

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. What is the function of a tokenizer in an LLM application?

A tokenizer converts input text into smaller units called tokens, which may be words, subwords, or characters. These tokens are converted into numerical IDs that the Large Language Model can process. Tokenization helps the model understand and generate text efficiently. For example, a sentence is split into tokens before being passed to the model. After generating a response, the token IDs are converted back into readable text. Thus, tokenization is an important step in processing text in LLM applications.

2. How does a Transformer-based LLM generate a response to a prompt?

A Transformer-based LLM first converts the input prompt into tokens and numerical representations. Its self-attention mechanism identifies relationships between tokens and understands the context. The model then predicts the probability of the next token based on the previous tokens. It repeatedly selects or samples tokens until a complete response is generated or a stopping condition is reached. Finally, the generated tokens are converted into readable text. This process enables the model to produce relevant, coherent, and context-aware responses to user prompts.

3. What is the purpose of Streamlit or Gradio in this experiment?

Streamlit and Gradio are Python frameworks used to create interactive user interfaces for AI applications. In this experiment, they provide a simple interface where users can enter prompts, submit questions, and view chatbot responses. Streamlit is suitable for building interactive dashboards and applications, while Gradio is particularly useful for demonstrating machine learning models. Both frameworks reduce the need for extensive HTML, CSS, and JavaScript coding. They make it easier to test, demonstrate, and interact with an LLM-based chatbot.

4. Why should an API key be stored securely and not committed to GitHub?

An API key is a secret credential that allows an application to access an external service. If it is publicly committed to GitHub, unauthorized users may copy and misuse it, potentially causing unexpected charges, data exposure, or service restrictions. Therefore, API keys should be stored securely using environment variables or secret-management tools. In Streamlit applications, secrets can be managed through the secrets configuration. Keeping credentials outside public source code improves application security and prevents unauthorized access to the services used by the chatbot.

5. What is conversation history, and how can it improve a chatbot?

Conversation history is the record of previous messages exchanged between a user and a chatbot. It includes earlier questions and responses that provide context for the current conversation. By using conversation history, a chatbot can understand follow-up questions, remember relevant details within the conversation, and provide more consistent responses. For example, if a user asks about Python and then says, "Give me an example," the chatbot can understand the reference. However, the history must be managed carefully to control token usage and protect user privacy.

6. Mention two limitations of LLM-based chatbots.

Two major limitations of LLM-based chatbots are hallucinations and limited contextual understanding. Hallucinations occur when a chatbot generates incorrect or fabricated information while presenting it confidently. This can lead to misleading answers. Limited contextual understanding means the chatbot may misunderstand ambiguous questions, lose track of earlier information, or struggle with very long conversations. Additionally, LLMs may produce biased responses because of their training data. Therefore, important outputs should be verified, and chatbots should not be treated as completely reliable sources of information.

7. How can you test whether a chatbot's responses are relevant?

A chatbot's responses can be tested by asking a variety of questions and comparing its answers with expected or verified information. Testing should check whether the response directly addresses the question, is factually correct, and follows the requested format. Follow-up questions can also test whether the chatbot maintains conversational context. Relevance can be evaluated using human feedback, predefined test cases, and automated evaluation metrics. Testing normal, ambiguous, and unexpected inputs helps identify weaknesses and improve the chatbot's reliability and overall performance.

8. What is the difference between running an application locally and deploying it online?
Running an application locally means executing it on a personal computer using a local development environment. It is mainly used for development, debugging, and testing, and access is generally limited to that computer or local network. Deploying an application online means hosting it on a server or cloud platform so users can access it through the internet. Online deployment requires suitable hosting, configuration, and security measures. Local execution offers convenient development, while online deployment enables wider access and sharing without requiring users to run the application themselves.

SUGGESTIONS FOR DATASETS

- Hugging Face Transformers documentation – https://huggingface.co/docs/transformers/index
- Hugging Face conversational / text-generation models – https://huggingface.co/models?pipeline_tag=text-generation
- Streamlit documentation – https://docs.streamlit.io/
- Gradio documentation – https://www.gradio.app/docs

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	